

Task Tracker

1. Document purpose

Task Tracker is a Django-based task-management application. This document describes the implemented design from the browser request through routing, authentication, application logic, validation, persistence, testing, and deployment configuration.

This guide is written for developers, reviewers, operators, and anyone who needs to install, understand, maintain, or extend the project.

2. Product overview

The application enables an authenticated user to:

- ? Create, view, edit, and delete tasks.
- ? Assign a task to a person.
- ? Set a priority: `LOW`, `MEDIUM`, or `HIGH`.
- ? Track a status: `TO\_DO`, `IN\_PROGRESS`, or `DONE`.
- ? Set a due date that cannot be earlier than today.
- ? Filter and sort tasks from the dashboard.
- ? Search, paginate, and manage tasks through a JSON API.
- ? Update profile information and change the account password.

Staff users also have a custom administration dashboard that displays user and task information.

3. Technology and runtime stack

Layer	Implemented technology
---	---
Language	Python
Framework	Django 5.2
Persistence	SQLite by default; MySQL configuration supported
Web rendering	Django templates with server-side HTML
API	Django function-based JSON endpoints
Authentication	Django `User`, sessions, and password hashing
Middleware	Sessions, CSRF, authentication, messages, and security
Testing	Django `TestCase`
Runtime entry points	WSGI and ASGI
Dependency manifest	`requirements.txt`

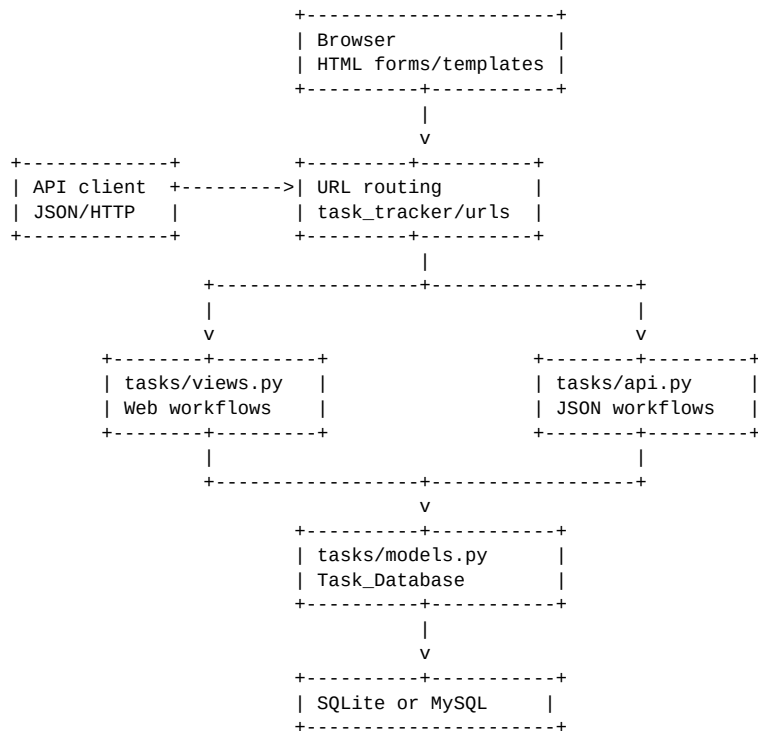
The project currently depends on:

Django>=5.2, <6.0

Install `mysqlclient` separately when MySQL is used.

4. High-level architecture

The application follows Django's model-template-view pattern and exposes two application surfaces: a server-rendered web UI and a JSON API.



4.1 Request lifecycle

1. The client sends an HTTP request.
2. Django middleware processes security, sessions, authentication, and messages.
3. `task\_tracker/urls.py` resolves project-level routes.
4. `tasks/urls.py` resolves task, profile, signup, and API routes.
5. `@login\_required` or a staff check protects restricted workflows.
6. The selected view validates input and calls the model layer.
7. Django's ORM reads or writes the configured database.
8. HTML views return templates; API views return JSON and HTTP status codes.

4.2 Application boundaries

? `task\_tracker/` is the Django project configuration package.

- ? `tasks/` is the application package containing business workflows.
- ? `templates/` is the presentation layer.
- ? `migrations/` is the versioned database schema history.
- ? `docs/screenshots/` contains UI reference images.

5. Repository structure

```
task_tracker/
  manage.py
  requirements.txt
  README.md
  TASK_TRACKER_DOCUMENTATION.pdf
  db.sqlite3
  task_tracker/
    __init__.py
    settings.py
    urls.py
    asgi.py
    wsgi.py
  tasks/
    __init__.py
    admin.py
    api.py
    apps.py
    models.py
    tests.py
    urls.py
    views.py
    migrations/
    templates/
      admin_access_denied.html
      admin_dashboard.html
      admin_login.html
      task_tracker_dashboard.html
      registration/
        login.html
        profile.html
        signup.html
  docs/
    screenshots/
```

Important files

File	Responsibility
---	---
`manage.py`	Django command-line entry point
`task_tracker/settings.py`	Installed apps, middleware, database, auth, and static settings
`task_tracker/urls.py`	Site-level URL composition
`tasks/models.py`	Task schema, choices, and due-date validation
`tasks/views.py`	HTML pages and form-based workflows
`tasks/api.py`	JSON serialization, validation, filtering, and CRUD
`tasks/urls.py`	App-level URL declarations
`tasks/tests.py`	API and dashboard regression tests
`tasks/migrations/`	Database schema transitions
`tasks/templates/`	User interface templates

6. Domain and data design

6.1 Django user

Authentication uses Django's built-in `auth.User` model. Django manages:

- ? Username and account identity.
- ? Password hashing and password validation.
- ? Staff and superuser flags.
- ? Login sessions.
- ? Last login and date-joined metadata.

The signup workflow creates users with `set_password()`, so raw passwords are not stored by the application.

6.2 `Task_Database` entity

The application task entity is defined in `tasks/models.py`.

Field	Database type	Required	Description
<code>id</code>	<code>BigAutoField</code>	Yes	Automatically generated primary key
<code>created_by</code>	<code>CharField(150)</code>	No	Username associated with the task
<code>task_name</code>	<code>CharField(100)</code>	Yes	Short task title
<code>task_description</code>	<code>CharField(500)</code>	No	Optional task details
<code>task_assignee</code>	<code>CharField(50)</code>	Yes	Person responsible for the task
<code>task_priority</code>	Choice field	Yes	<code>LOW</code> , <code>MEDIUM</code> , or <code>HIGH</code>
<code>task_status</code>	Choice field	Yes	Defaults to <code>TO_DO</code>
<code>task_due_date</code>	<code>DateField</code>	Yes	Today or a future date
<code>task_created_at</code>	<code>DateTimeField</code>	Yes	Set automatically at creation

The model's `clean()` method rejects a due date earlier than `timezone.localdate()`. Both the HTML and API workflows call `full_clean()` before saving, so the same model rule is applied to both entry points.

6.3 Ownership strategy

The current schema stores the creator as a username string instead of a database foreign key. The dashboard and API list/detail queries use that string to identify a user's tasks.

For a production-scale system, the recommended design is:

```
created_by = models.ForeignKey(
    User,
    on_delete=models.CASCADE,
    related_name="tasks",
)
```

This would preserve referential integrity and continue working if a username changes.

7. Authentication and authorization

7.1 Normal user authentication

- ? ``/`` and ``/login/`` display the login form.
- ? ``login_page`` authenticates credentials with Django.
- ? Successful authentication creates a session and redirects to ``/dashboard/``.
- ? ``/logout/`` uses Django's ``LogoutView``.
- ? Unauthenticated protected requests are redirected to the configured login route.

7.2 Signup

``sign_up`` accepts username, email, password, and password confirmation. It checks required fields, confirms matching passwords, checks username uniqueness, hashes the password, and redirects to login after creation.

7.3 Profile and password management

``profile`` is protected by ``@login_required``.

- ? Profile updates change first name, last name, and email.
- ? Password changes use Django's ``PasswordChangeForm``.
- ? ``update_session_auth_hash()`` keeps the user signed in after changing the password.

7.4 Staff administration

The custom administration flow has two layers:

1. ``admin_login_page`` authenticates credentials and requires ``is_staff=True``.
2. ``admin_dashboard`` applies ``@login_required(login_url="admin_login")`` and performs a second staff check.

The standard Django admin remains available under ``/admin/`` and uses Django's normal staff, superuser, and permission model.

7.5 Authorization review notes

The dashboard query and API endpoints restrict reads to the current user's username. The current HTML edit and delete views authenticate the request but fetch a task by ID without also filtering by ``created_by``. A production hardening change should scope those lookups to the current user, for example:

```
get_object_or_404(
    Task_Database,
```

```
        id=task_id,  
        created_by=request.user.username,  
    )
```

This note documents the current implementation and the recommended security improvement; it does not silently change application behavior.

8. Web application design

8.1 Login and signup screens

The login, signup, admin login, profile, and dashboard pages are Django templates. Forms use POST requests and include CSRF tokens where implemented. The Django messages framework displays success and validation feedback.

8.2 Dashboard

``task_tracker_dashboard``:

1. Starts with tasks created by the signed-in user.
2. Applies an optional ``status`` filter.
3. Applies an optional ``priority`` filter.
4. Sorts by due date in ascending or descending order.
5. Calculates total, to-do, in-progress, and completed counts.
6. Renders ``task_tracker_dashboard.html``.

Example:

```
/dashboard/?status=IN_PROGRESS&priority=HIGH&sort=desc
```

8.3 Create task

``create_task`` reads form fields, associates the task with the current username, leaves status unset so the model default is used, validates with ``full_clean()``, saves the record, and redirects back to the dashboard.

8.4 Edit task

``edit_task`` reads task fields from the POST body, including status, validates the complete model, saves the task, displays a message, and redirects to the dashboard.

8.5 Delete task

``delete_task`` accepts POST, deletes the selected record, displays a success message, and redirects to the dashboard.

9. URL and page catalogue

Method	URL	Handler	Access
---	---	---	---
`GET`, `POST`	`/`	`login_page`	Public
`GET`, `POST`	`/login/`	`login_page`	Public
`GET`, `POST`	`/signup/`	`sign_up`	Public
`GET`, `POST`	`/profile/`	`profile`	Authenticated
`GET`	`/dashboard/`	`task_tracker_dashboard`	Authenticated
`POST`	`/task/create/`	`create_task`	Authenticated
`POST`	`/task/edit/<id>/`	`edit_task`	Authenticated
`POST`	`/task/delete/<id>/`	`delete_task`	Authenticated
`GET`, `POST`	`/admin-dashboard/login/`	`admin_login_page`	Public
`GET`	`/admin-dashboard/`	`admin_dashboard`	Staff
`POST`	`/logout/`	Django `LogoutView`	Authenticated
`GET`, `POST`	`/admin/`	Django admin	Staff/admin

10. JSON API design

All API endpoints use Django session authentication and are protected with `@login\_required`.

10.1 Endpoint summary

Method	Endpoint	Behavior	Success
---	---	---	---
`GET`	`/api/tasks/`	List current user's tasks	`200`
`POST`	`/api/tasks/`	Create a task	`201`
`GET`	`/api/tasks/<id>/`	Retrieve one owned task	`200`
`PUT`	`/api/tasks/<id>/`	Replace task fields	`200`
`PATCH`	`/api/tasks/<id>/`	Partially update a task	`200`
`DELETE`	`/api/tasks/<id>/`	Delete one owned task	`204`

10.2 Create and update fields

Allowed fields are:

```
task_name
task_description
task_assignee
task_priority
task_status
task_due_date
```

For `POST` and `PUT`, these fields are required:

```
task_name
task_assignee
task_priority
task_due_date
```

Unknown fields are rejected. String values are trimmed before assignment.

Model validation is executed before persistence.

Example request:

```
POST /api/tasks/
Content-Type: application/json

{
  "task_name": "Prepare release notes",
  "task_description": "Summarize the next release",
  "task_assignee": "Dheeraj",
  "task_priority": "MEDIUM",
  "task_due_date": "2026-10-01"
}
```

10.3 List filters and pagination

Parameter	Values	Purpose
---	---	---
<code>`status`</code>	<code>`TO_DO`</code> , <code>`IN_PROGRESS`</code> , <code>`DONE`</code>	Filter status
<code>`priority`</code>	<code>`LOW`</code> , <code>`MEDIUM`</code> , <code>`HIGH`</code>	Filter priority
<code>`q`</code>	Text	Search name, description, or assignee
<code>`sort`</code>	<code>`asc`</code> , <code>`desc`</code>	Sort by due date
<code>`page`</code>	Positive integer	Page number
<code>`limit`</code>	<code>`1`</code> to <code>`100`</code>	Page size

Without ``page`` or ``limit``:

```
{
  "tasks": [
    {
      "id": 1,
      "created_by": "dheeraj",
      "task_name": "Prepare release notes",
      "task_description": "Summarize the next release",
      "task_assignee": "Dheeraj",
      "task_priority": "MEDIUM",
      "task_status": "TO_DO",
      "task_due_date": "2026-10-01",
      "task_created_at": "2026-09-23T07:30:00+00:00"
    }
  ]
}
```

When pagination is requested, the response includes:

```
{
  "tasks": [],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 42,
    "pages": 3
  }
}
```

10.4 Error contract

General errors use:

```
{
  "detail": "Invalid status filter."
}
```

```
}
```

Model validation errors use:

```
{
  "detail": "Validation failed.",
  "errors": {
    "task_due_date": [
      "Due date must be today or a future date."
    ]
  }
}
```

Malformed JSON, unknown fields, missing required fields, invalid filters, invalid pagination, unsupported methods, and missing tasks are handled explicitly by the API.

10.5 API security note

The current API views use `csrf\_exempt` together with Django session authentication. This is convenient for clients sending JSON but requires a careful deployment review. For browser-based session clients, enable and validate CSRF tokens or use a dedicated token-based authentication strategy before exposing the API outside a trusted environment.

11. Database and migrations

SQLite is the default database:

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": "db.sqlite3",
    }
}
```

The settings select the database engine and connection values from environment variables. Existing migrations are stored under `tasks/migrations/` and should be applied with:

```
python manage.py migrate
```

When the model changes:

```
python manage.py makemigrations
python manage.py migrate
```

Never edit an already-applied migration to change history. Create a new migration instead.

12. Windows installation

12.1 Prerequisites

Install:

- ? Python 3.10 or newer.
- ? Git.
- ? MySQL Server 8.0 or newer when MySQL is required.

12.2 Create and activate the environment

```
cd C:\Users\Dheeraj\Desktop\demoproject\task_tracker
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

If PowerShell blocks activation:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
.\.venv\Scripts\Activate.ps1
```

12.3 Initialize and run

```
python manage.py migrate
python manage.py check
python manage.py runserver
```

Open <<http://127.0.0.1:8000/>>.

Create an administrator when required:

```
python manage.py createsuperuser
```

13. MySQL configuration

13.1 Create the database and user

```
CREATE DATABASE task_tracker
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

CREATE USER 'task_tracker_user'@'localhost'
  IDENTIFIED BY 'replace-with-a-strong-password';

GRANT ALL PRIVILEGES ON task_tracker.*
  TO 'task_tracker_user'@'localhost';

FLUSH PRIVILEGES;
```

13.2 Install the driver

```
.\.venv\Scripts\Activate.ps1
pip install mysqlclient
```

If the driver needs to compile on Windows, install Microsoft C++ Build Tools with the **Desktop development with C++** workload.

13.3 Configure PowerShell

```
$env:DB_ENGINE = "django.db.backends.mysql"
$env:DB_NAME = "task_tracker"
$env:DB_USER = "task_tracker_user"
$env:DB_PASSWORD = "replace-with-a-strong-password"
$env:DB_HOST = "127.0.0.1"
$env:DB_PORT = "3306"
```

Then verify:

```
python manage.py check
python manage.py migrate
```

These variables apply only to the current PowerShell window. Do not commit database passwords or secret values.

14. Configuration and deployment

The checked-in settings are development settings. Before production:

1. Move `SECRET\_KEY` to a secret manager and rotate the development value.
2. Set `DEBUG=False`.
3. Configure `ALLOWED\_HOSTS`.
4. Use a production database and apply migrations during deployment.
5. Configure static-file collection and a production web server.
6. Use HTTPS and secure cookie settings.
7. Configure CSRF trusted origins for the deployed domains.
8. Configure structured logging and error monitoring.
9. Back up the database and test restoration.
10. Review the HTML task ownership issue described above.
11. Review the API CSRF/authentication strategy.
12. Run Django deployment checks:

```
python manage.py check --deploy
```

The repository contains WSGI and ASGI entry points. A production server can use `task\_tracker.wsgi:application` for WSGI or `task\_tracker.asgi:application` for ASGI.

15. Testing strategy

Run:

```
python manage.py check
python manage.py test
```

The current test suite covers:

- ? API task creation and default status.
- ? API listing.

- ? API update and deletion.
- ? Due-date validation.
- ? Search and pagination.
- ? Invalid status and pagination parameters.
- ? Malformed JSON.
- ? Basic public page availability.

Recommended future tests include:

- ? Unauthenticated web and API access.
- ? Staff versus non-staff admin access.
- ? Cross-user task access.
- ? HTML create, edit, and delete workflows.
- ? Profile and password changes.
- ? CSRF behavior.
- ? MySQL integration in a CI environment.

Before committing:

```
git diff --check
python manage.py check
python manage.py test
```

16. Screenshots and documentation assets

UI screenshots are stored in

docs/screenshots. The printable version of this document is TASK_TRACKER_DOCUMENTATION.pdf.

17. Extension and maintenance workflow

For a new feature:

1. Define or update the data model.
2. Create a migration.
3. Add model validation where the rule belongs to the domain.
4. Add the HTML view and template.
5. Add or update the API contract if needed.
6. Add URL routes.
7. Add regression tests.
8. Update this README and regenerate the PDF.
9. Run checks and tests.
10. Review authorization, CSRF, input validation, and error handling.

Keep business rules in the model when they apply to every entry point. Keep transport-specific parsing and response formatting in the API layer. Reuse Django authentication, password hashing, session, and CSRF mechanisms rather than introducing custom security primitives.

18. Design roadmap

The following improvements are recommended as the project grows:

- ? Replace `created_by` with a `ForeignKey` to `User`.
- ? Enforce owner filtering in every HTML task lookup.
- ? Add database indexes for ownership, status, priority, and due date.
- ? Introduce Django forms for task input validation and error rendering.
- ? Add token authentication or an explicit CSRF strategy for external API consumers.
- ? Move inline CSS into versioned static files.
- ? Add pagination to the HTML dashboard for large task collections.
- ? Add service-layer functions when workflows become more complex.
- ? Add CI checks for migrations, tests, security checks, and formatting.
- ? Separate development and production settings.